

安徽大学《深度学习与神经网络》

实验报告 5

学号： WA2224013 专业： 机器人工程 姓名： 郭义月
实验日期： 2025.6.15 教师签字： _____ 成绩： _____

[实验名称] 注意力机制与 Transformer 实验

[实验目的]

1. 熟悉和掌握注意力评分函数、Bahdanau 注意力、多头注意力
2. 熟悉和掌握自注意力和位置编码
3. 熟悉和掌握 Transformer

[实验要求]

1. 采用 Python 语言基于 PyTorch 深度学习框架进行编程
2. 代码可读性强：变量、函数、类等命名可读性强，包含必要的注释
3. 提交实验报告要求：
 - 命名方式：“学号-姓名-Lab-N” (N 为实验课序号，即：1-6)；
 - 截止时间：下次实验课当晚 23:59；
 - 提交方式：智慧安大-网络教育平台-作业；

按时提交（**过时不补**）；

[实验内容]

1. 多种注意力评分函数

- 学习、运行、调试和比较参考教材 10.3 小节多种类型注意力评分函数
- 完成练习题 1

2. Bahdanau 注意力:

- 学习、运行和调试参考教材 10.4 小节内容

3. 多头注意力:

- 学习、运行和调试参考教材 10.5 小节内容
- 完成练习题 1

4. 自注意力和位置编码:

- 学习、运行和调试参考教材 10.6 小节内容
- 完成练习题 2

5. Transformer:

- 学习、运行和调试参考教材 10.7 小节内容
- 完成练习题 3 和 6

6. 参考资料:

- 参考教材: <https://zh-v2.d2l.ai/d2l-zh-pytorch.pdf>
- PyTorch 官方文档: <https://pytorch.org/docs/2.0/>;
- PyTorch 官方论坛: <https://discuss.pytorch.org/>

[实验代码和结果]

实验 1：多种注意力评分函数，学习 10.3

10.3.1 掩蔽 softmax 操作

softmax 操作用于输出一个概率分布作为注意力权重。在某些情况下，并非所有的值都应该被纳入到注意力汇聚中。例如，为了高效处理小批量数据集，某些文本序列被填充了没有意义的特殊词元。为了仅将有意义的词元作为值来获取注意力汇聚，可以指定一个有效序列长度（即词元的个数），以便在计算 softmax 时过滤掉超出指定范围的位置。masked_softmax 函数实现了这样的掩蔽 softmax 操作（masked softmax operation），其中任何超出有效长度的位置都被掩蔽并置为 0。

掩蔽softmax操作，任何超出有效长度的位置都将被掩蔽并置为0

```
def masked_softmax(X, valid_lens):
    if valid_lens is None:
        return nn.functional.softmax(X, dim=-1)
    else:
        shape = X.shape
        if valid_lens.dim() == 1:
            valid_lens = torch.repeat_interleave(valid_lens, shape[1])
        else:
            valid_lens = valid_lens.reshape(-1)
        X = d2l.sequence_mask(X.reshape(-1, shape[-1]), valid_lens, value=-1e6)
        return nn.functional.softmax(X.reshape(shape), dim=-1)
```

对于由两个 2×4 矩阵表示的样本，这两个样本的有效长度分别为 2 和 3。经过掩蔽 softmax 操作，超出有效长度的值都被掩蔽为 0

```
masked_softmax(torch.rand(2,2,4), torch.tensor([2, 3]))

tensor([[[0.6411, 0.3589, 0.0000, 0.0000],
         [0.6249, 0.3751, 0.0000, 0.0000]],

        [[0.3294, 0.2984, 0.3721, 0.0000],
         [0.4742, 0.2157, 0.3101, 0.0000]]])
```

也可以使用二维张量，为矩阵样本中的每一行指定有效长度。

```
masked_softmax(torch.rand(2,2,4), torch.tensor([[1,3], [2,4]]))

tensor([[[1.0000, 0.0000, 0.0000, 0.0000],
         [0.3169, 0.2615, 0.4216, 0.0000]],

        [[0.6439, 0.3561, 0.0000, 0.0000],
         [0.1681, 0.4424, 0.2062, 0.1832]]])
```

10.3.2 加性注意力

当查询和键是不同长度的矢量时，可以使用加性注意力作为评分函数，实现加性注意力的代码为：

加性注意力

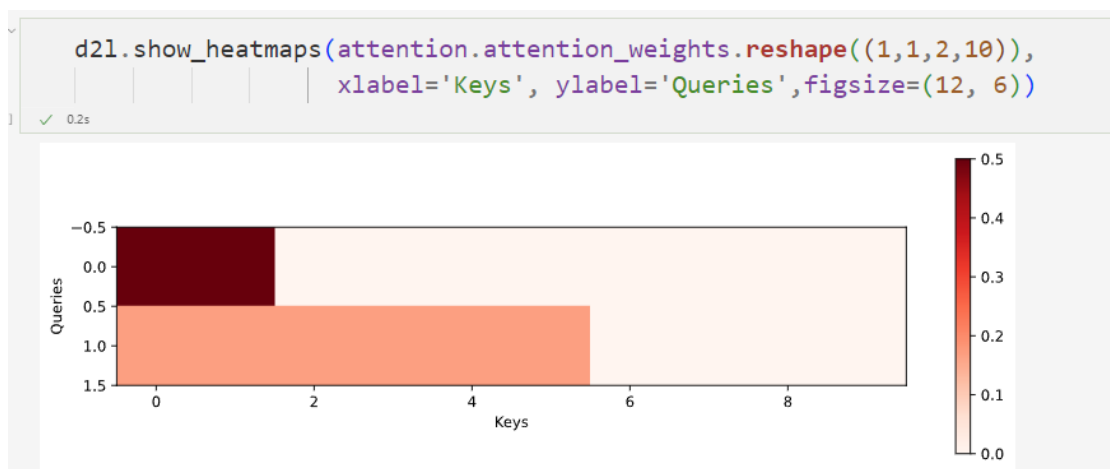
```
class AdditiveAttention(nn.Module):
    def __init__(self, key_size, query_size, num_hiddens, dropout, **kwargs):
        super(AdditiveAttention, self).__init__(**kwargs)
        self.W_k = nn.Linear(key_size, num_hiddens, bias=False)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=False)
        self.W_v = nn.Linear(num_hiddens, 1, bias=False)
        self.dropout = nn.Dropout(dropout)
    def forward(self, queries, keys, values, valid_lens):
        queries, keys = self.W_q(queries), self.W_k(keys)
        # 在维度扩展后, queries的形状: (batch_size, 查询的个数, 1, num_hidden)
        # key的形状: (batch_size, 1, 键值对的个数, num_hiddens)
        # 使用广播方式求和
        features = queries.unsqueeze(2) + keys.unsqueeze(1)
        features = torch.tanh(features)
        scores = self.W_v(features).squeeze(-1)
        self.attention_weights = masked_softmax(scores, valid_lens)
        return torch.bmm(self.dropout(self.attention_weights), values)
```

当查询、键和值的形状为（批量大小，步数或词元序列长度，特征大小）时，实际输出为(2, 1, 20)、(2, 10, 2)和(2, 10, 4)。注意力汇聚输出的形状为（批量大小，查询的步数，值的维度）。

```
queries, keys = torch.normal(0, 1, (2, 1, 20)), torch.ones((2, 10, 2))
values = torch.arange(40, dtype=torch.float32).reshape(1, 10, 4).repeat(2, 1, 1)
valid_lens = torch.tensor([2, 6])
attention = AdditiveAttention(key_size=2, query_size=20, num_hiddens=8, dropout=0.1)
attention.eval()
attention(queries, keys, values, valid_lens)

tensor([[[ 2.0000,  3.0000,  4.0000,  5.0000]],
        [[10.0000, 11.0000, 12.0000, 13.0000]]], grad_fn=<BmmBackward0>)
```

由于每个键都是相同的，所以注意力权重是均匀的，由指定的有效长度决定。



10.3.3 缩放点积注意力

使用点积可以得到计算效率更高的评分函数，但是点积操作要求查询和键具有相同的长度 d 。假设查询和键的所有元素都是独立的随机变量，并且都满足零均值和单位方差，那么两个向量的点积的均值为 0，方差为 d 。

缩放点积注意力的评分函数为：

$$a(q,k) = q^T k / \sqrt{d}$$

下面的缩放点积注意力的实现使用了暂退法进行模型正则化。

缩放点积注意力

```
class DotProductAttention(nn.Module):
    def __init__(self, dropout, **kwargs):
        super(DotProductAttention, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout)

    def forward(self, queries, keys, values, valid_lens=None):
        d = queries.shape[-1]
        scores = torch.bmm(queries, keys.transpose(1,2))/math.sqrt(d)
        self.attention_weights = masked_softmax(scores, valid_lens)
        return torch.bmm(self.dropout(self.attention_weights), values)
```

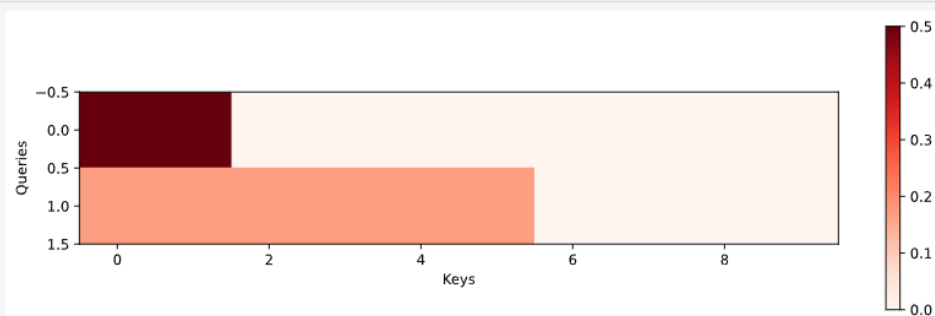
使用与先前加性注意力例子中相同的键、值和有效长度。对于点积操作，我们令查询的特征维度与键的特征维度大小相同。

```
queries = torch.normal(0, 1, (2, 1, 2))
attention = DotProductAttention(dropout=0.5)
attention.eval()
attention(queries, keys, values, valid_lens)

tensor([[[ 2.0000,  3.0000,  4.0000,  5.0000]],
        [[10.0000, 11.0000, 12.0000, 13.0000]])])
```

与加性注意力演示相同，由于键包含的是相同的元素，而这些元素无法通过任何查询进行区分，因此获得了均匀的注意力权重。

```
d2l.show_heatmaps(attention.attention_weights.reshape((1,1,2,10)),
                  xlabel='Keys', ylabel='Queries', figsize=(12, 6))
```



实验 2: Bahdanau 注意力, 学习 10.4

10.4.1 模型

Bahdanau 注意力模型

$$c_{t'} = \sum_{t=0}^T \alpha(s_{t'-1}, h_t) h_t$$

时间步 $t' - 1$ 时的解码器隐状态 $s_{t' - 1}$ 是查询, 编码器隐状态 h_t 既是键, 也是值, 注意力权重 α 是加性注意力打分函数计算的。

10.4.2 定义注意力解码器

定义注意力解码器实现循环神经网络编码-解码器

定义注意力解码器

```
class AttentionDecoder(d2l.Decoder):
    """带有注意力机制解码器的基本接口"""
    def __init__(self, **kwargs):
        super(AttentionDecoder, self).__init__(**kwargs)

    @property
    def attention_weights(self):
        raise NotImplementedError
```

初始化解码器的状态, 需要下面的输入:

1. 编码器在所有时间步的最终层隐状态, 将作为注意力的键和值;
2. 上一时间步的编码器全层隐状态, 将作为初始化解码器的隐状态;
3. 编码器有效长度 (排除在注意力池中填充词元)。

在每个解码时间步骤中, 解码器上一个时间步的最终层隐状态将用作查询。因此, 注意力输出和输入嵌入都连结为循环神经网络解码器的输入。

```

class Seq2SeqAttentionDecoder(AttentionDecoder):
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers, dropout=0, **kwargs):
        super(Seq2SeqAttentionDecoder, self).__init__(**kwargs)
        self.attention = d2l.AdditiveAttention(num_hiddens, num_hiddens, num_hiddens, dropout)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(
            embed_size+num_hiddens, num_hiddens, num_layers, dropout=dropout
        )
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, enc_valid_lens, *arg):
        outputs, hidden_state = enc_outputs
        return (outputs.permute(1,0,2), hidden_state, enc_valid_lens)

    def forward(self, X, state):
        enc_outputs, hidden_state, enc_valid_lens = state
        X = self.embedding(X).permute(1,0,2)
        outputs, self._attention_weights = [], []
        for x in X:
            query = torch.unsqueeze(hidden_state[-1], dim=1)
            context = self.attention(query, enc_outputs, enc_outputs, enc_valid_lens)
            x = torch.cat((context, torch.unsqueeze(x, dim=1)), dim=-1)
            out, hidden_state = self.rnn(x.permute(1, 0, 2), hidden_state)
            outputs.append(out)
            self._attention_weights.append(self.attention.attention_weights)
        outputs = self.dense(torch.cat(outputs, dim=0))
        return outputs.permute(1,0,2), [enc_outputs, hidden_state, enc_valid_lens]

    @property
    def attention_weight(self):
        return self._attention_weights

```

使用包含 7 个时间步的 4 个序列输入的小批量测试 Bahdanau 注意力解码器。

```

encoder = d2l.Seq2SeqEncoder(vocab_size=10, embed_size=8, num_hiddens=16, num_layers=2)
encoder.eval()
decoder = Seq2SeqAttentionDecoder(vocab_size=10, embed_size=8, num_hiddens=16, num_layers=2)
decoder.eval()
X = torch.zeros((4,7), dtype=torch.long)
state = decoder.init_state(encoder(X), None)
output, state = decoder(X, state)
output.shape, len(state), state[0].shape, len(state[1]), state[1][0].shape

(torch.Size([4, 7, 10]), 3, torch.Size([4, 7, 16]), 2, torch.Size([4, 16]))

```

10.4.3 训练

指定超参数实例化一个带有 Bahdanau 注意力的编码器和解码器，并对这个模型进行机器翻译训练

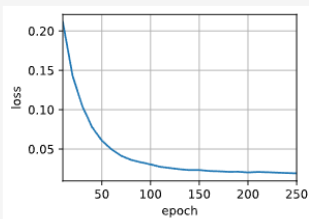
```

embed_size, num_hiddens, num_layers, dropout = 32, 32, 2, 0.1
batch_size, num_steps = 64, 10
lr, num_epochs, device = 0.005, 250, d2l.try_gpu()

train_iter, src_vocab, tgt_vocab = d2l.load_data_nmt(batch_size, num_steps)
encoder = d2l.Seq2SeqEncoder(
    len(src_vocab), embed_size, num_hiddens, num_layers, dropout)
decoder = Seq2SeqAttentionDecoder(
    len(tgt_vocab), embed_size, num_hiddens, num_layers, dropout)
net = d2l.EncoderDecoder(encoder, decoder)
d2l.train_seq2seq(net, train_iter, lr, num_epochs, tgt_vocab, device)

```

loss 0.019, 5342.5 tokens/sec on cuda:0



模型训练后用它将几个英语句子翻译成法语并计算它们的 BLEU 分数。

```

engs = ['go .', "i lost .", 'he\'s calm .', 'i\'m home .']
fras = ['va !', 'j\'ai perdu .', 'il est calme .', 'je suis chez moi .']
for eng, fra in zip(engs, fras):
    translation, dec_attention_weight_seq = d2l.predict_seq2seq(
        net, eng, src_vocab, tgt_vocab, num_steps, device, True)
    print(f'{eng} => {translation}, ',
          f'bleu {d2l.bleu(translation, fra, k=2):.3f}')

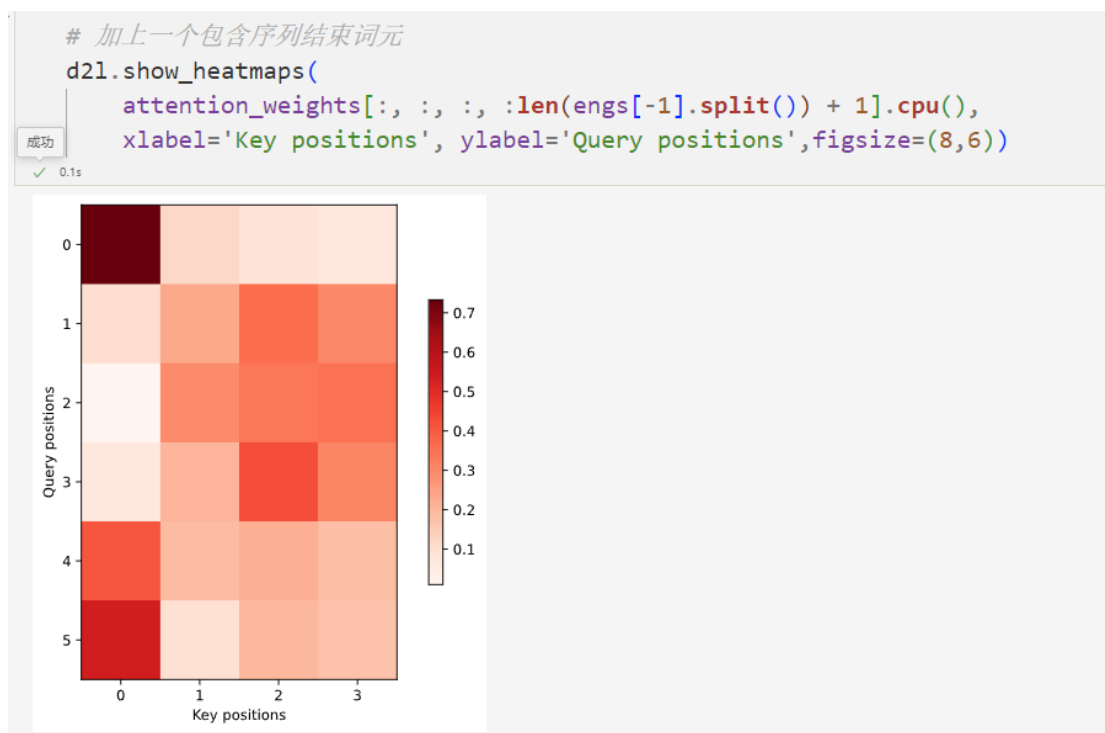
```

```

go . => va !, bleu 1.000
i lost . => j'ai perdu ., bleu 1.000
he's calm . => je vais bien ., bleu 0.000
i'm home . => je suis chez moi ., bleu 1.000

```

训练结束后，下面通过可视化注意力权重会发现，每个查询都会在键值对上分配不同的权重，这说明在每个解码步中，输入序列的不同部分被选择性地聚集在注意力池中。



实验 3：多头注意力，学习 10.5

10.5.1 模型

每个注意力头的计算方法为

$$h_i = f\left(W_i^{(q)}q, W_i^{(k)}k, W_i^{(v)}v\right) \in R^{p_v}$$

其可学习参数是 W_o ，每个头都可能会关注输入的不同部分，可以表示比简单加权平均值更复杂的函数。

10.5.2 实现

在实现过程中通常选择缩放点积注意力作为每一个注意力头。为了避免计算代价和参数代价的大幅增长，设定 $pq=pk=pv=po/h$ 。如果将查询、键和值的线性变换的输出数量设置为 $pqh=pkh=pvh=po$ ，则可以并行计算 h 个头。在下面的实现中， po 是通过参数 `num_hiddens` 指定的。

```

class MultiHeadAttention(nn.Module):
    def __init__(self, key_size, query_size, value_size, num_hiddens, num_heads, dropout, bias=False, **kwargs):
        super(MultiHeadAttention, self).__init__(**kwargs)
        self.num_heads = num_heads
        self.attention = d2l.DotProductAttention(dropout)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=bias)
        self.W_k = nn.Linear(key_size, num_hiddens, bias=bias)
        self.W_v = nn.Linear(value_size, num_hiddens, bias=bias)
        self.W_o = nn.Linear(num_hiddens, num_hiddens, bias=bias)

    def forward(self, queries, keys, values, valid_lens):
        queries = transpose_qkv(self.W_q(queries), self.num_heads)
        keys = transpose_qkv(self.W_k(keys), self.num_heads)
        values = transpose_qkv(self.W_v(values), self.num_heads)
        if valid_lens is not None:
            valid_lens = torch.repeat_interleave(valid_lens, repeats=self.num_heads, dim=0)
        output = self.attention(queries, keys, values, valid_lens)
        output_concat = transpose_output(output, self.num_heads)
        return self.W_o(output_concat)

```

为了能够使多个头并行计算，上面的 MultiHeadAttention 类将使用下面定义的两个转置函数。具体来说，transpose_output 函数反转了 transpose_qkv 函数的操作。

```

def transpose_qkv(X, num_heads):
    X = X.reshape(X.shape[0], X.shape[1], num_heads, -1)
    X = X.permute(0, 2, 1, 3)
    return X.reshape(-1, X.shape[2], X.shape[3])

def transpose_output(X, num_heads):
    X = X.reshape(-1, num_heads, X.shape[1], X.shape[2])
    X = X.permute(0, 2, 1, 3)
    return X.reshape(X.shape[0], X.shape[1], -1)

```

使用键和值相同的小例子来测试我们编写的 MultiHeadAttention 类。

```

num_hiddens, num_heads = 100, 5
attention = MultiHeadAttention(num_hiddens, num_hiddens, num_hiddens, num_hiddens, num_heads, 0.5)
attention.eval()

MultiHeadAttention(
  (attention): DotProductAttention(
    (dropout): Dropout(p=0.5, inplace=False)
  )
  (W_q): Linear(in_features=100, out_features=100, bias=False)
  (W_k): Linear(in_features=100, out_features=100, bias=False)
  (W_v): Linear(in_features=100, out_features=100, bias=False)
  (W_o): Linear(in_features=100, out_features=100, bias=False)
)

batch_size, num_queries = 2, 4
num_kvpairs, valid_lens = 6, torch.tensor([3,2])
X = torch.ones((batch_size, num_queries, num_hiddens))
Y = torch.ones((batch_size, num_kvpairs, num_hiddens))
attention(X, Y, Y, valid_lens).shape

torch.Size([2, 4, 100])

```

实验 4：自注意力和位置编码 学习 10.6

10.6.1 自注意力

给定一个由词元组成的输入序列 $\mathbf{x}_1, \dots, \mathbf{x}_n$ ，其中任意 $\mathbf{x}_i \in \mathbb{R}^d (1 \leq i \leq n)$ 。该序列的自注意力输出为一个长

度相同的序列 $y_1, \dots, y_n$, 其中:

$$y_i = f(x_i, (x_1, x_1), \dots, (x_n, x_n))$$

基于多头注意力对一个张量完成自注意力的计算, 张量的形状为 (批量大小, 时间步的数目或词元序列的长度, d)。输出与输入的张量形状相同。

```
自注意力

num_hiddens, num_heads = 100, 5
attention = d2l.MultiHeadAttention(num_hiddens, num_hiddens, num_hiddens, num_hiddens, num_heads, 0.5)
attention.eval()

MultiHeadAttention(
  (attention): DotProductAttention(
    (dropout): Dropout(p=0.5, inplace=False)
  )
  (W_q): Linear(in_features=100, out_features=100, bias=False)
  (W_k): Linear(in_features=100, out_features=100, bias=False)
  (W_v): Linear(in_features=100, out_features=100, bias=False)
  (W_o): Linear(in_features=100, out_features=100, bias=False)
)

batch_size, num_queries, valid_lens = 2, 4, torch.tensor([3,2])
X = torch.ones(batch_size, num_queries, num_hiddens)
attention(X, X, X, valid_lens).shape

torch.Size([2, 4, 100])
```

10.6.3 位置编码

在处理词元序列时, 循环神经网络是逐个的重复地处理词元的, 而自注意力则因为并行计算而放弃了顺序操作。为了使用序列的顺序信息, 通过在输入表示中添加位置编码 (positional encoding) 来注入绝对的或相对的位置信息。

```
class PositionalEncoding(nn.Module):
    def __init__(self, num_hiddens, dropout, max_len=1000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(dropout)
        self.P = torch.zeros((1, max_len, num_hiddens))
        X = torch.arange(max_len, dtype=torch.float32).reshape(-1, 1)/torch.pow(10000, torch.arange(0, num_hiddens, 2, dtype=torch.float32)/num_hiddens)
        self.P[:, :, 0:2] = torch.sin(X)
        self.P[:, :, 1:2] = torch.cos(X)
    def forward(self, X):
        X = X + self.P[:, :, :X.shape[1], :].to(X.device)
        return self.dropout(X)
```

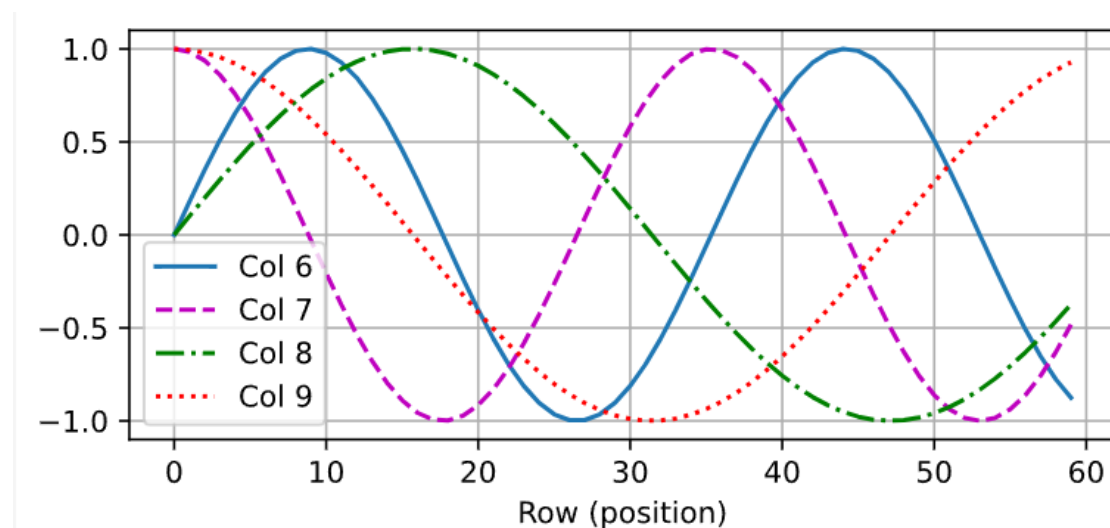
在位置嵌入矩阵 $\mathbf{P}$ 中, 行代表词元在序列中的位置, 列代表位置编码的不同维度。从下面的例子中可以看到位置嵌入矩阵的第 6 列和第 7 列的频率高于第 8 列和第 9 列。第 6 列和第 7 列之间的偏移量 (第 8 列和第 9 列相同) 是由于正弦函数和余弦函数的交替。

```

encoding_dim, num_steps = 32, 60
pos_encoding = PositionalEncoding(encoding_dim, 0)
pos_encoding.eval()
X = pos_encoding(torch.zeros((1, num_steps, encoding_dim)))
P = pos_encoding.P[:, :X.shape[1], :]
d21.plot(torch.arange(num_steps), P[0, :, 6:10].T,
          xlabel='Row (position)', figsize=(6, 2.5),
          legend=["Col %d" % d for d in torch.arange(6,10)])

```

结果为:



打印绝对位置信息:

```

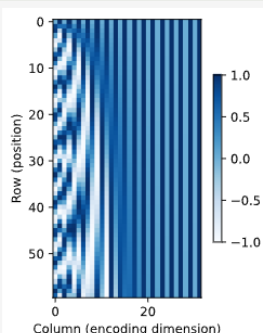
for i in range(8):
    print(f'{i} 的二进制是: {i: >30b}')

```

0 的二进制是:	0
1 的二进制是:	1
2 的二进制是:	10
3 的二进制是:	11
4 的二进制是:	100
5 的二进制是:	101
6 的二进制是:	110
7 的二进制是:	111

在二进制表示中, 较高比特位的交替频率低于较低比特位, 位置编码通过使用三角函数在编码维度上降低频率。由于输出是浮点数, 因此此类连续表示比二进制表示法更节省空间。

```
P = P[0, :, :].unsqueeze(0).unsqueeze(0)
d2l.show_heatmaps(P, xlabel='Column (encoding dimension)',
                  ylabel='Row (position)', figsize=(3.5, 4), cmap='Blues')
```



实验 5: Transformer, 学习 10.7

10.7.2 基于位置的前馈网络

基于位置的前馈网络对序列中的所有位置的表示进行变换时使用的是同一个多层感知机（MLP），这就是称前馈网络是基于位置的（**positionwise**）的原因。输入 X 的形状（批量大小，时间步数或序列长度，隐单元数或特征维度）将被一个两层的感知机转换形状为（批量大小，时间步数，`ffn_num_outputs`）的输出张量。

```
class PositionWiseFFN(nn.Module):
    def __init__(self, ffn_num_input, ffn_num_hiddens, ffn_num_outputs, **kwargs):
        super(PositionWiseFFN, self).__init__(**kwargs)
        self.dense1 = nn.Linear(ffn_num_input, ffn_num_hiddens)
        self.relu = nn.ReLU()
        self.dense2 = nn.Linear(ffn_num_hiddens, ffn_num_outputs)
    def forward(self, X):
        return self.dense2(self.relu(self.dense1(X)))
```

改变张量的最里层维度的尺寸，会改变成基于位置的前馈网络的输出尺寸。因为用同一个多层感知机对所有位置上的输入进行变换，所以当所有这些位置的输入相同时，它们的输出也是相同的。

```
ffn = PositionWiseFFN(4, 4, 8)
ffn.eval()
ffn(torch.ones((2, 3, 4)))[0]

tensor([[[-0.1956,  0.5872, -0.2146, -0.1668, -0.0449,  0.6440,  0.5845, -0.1267],
         [-0.1956,  0.5872, -0.2146, -0.1668, -0.0449,  0.6440,  0.5845, -0.1267],
         [-0.1956,  0.5872, -0.2146, -0.1668, -0.0449,  0.6440,  0.5845, -0.1267]]],
        grad_fn=<SelectBackward0>)
```

10.7.3 残差连接和层规范化

层规范化和批量规范化的目标相同，但层规范化是基于特征维度进行规范化。尽

管批量规范化在计算机视觉中被广泛应用，但在自然语言处理任务中（输入通常是变长序列）批量规范化通常不如层规范化的效果好。

```
>
ln = nn.LayerNorm(2)
bn = nn.BatchNorm1d(2)
X = torch.tensor([[1, 2], [2, 3]], dtype=torch.float32)
print('layer norm:', ln(X), '\nbatch norm:', bn(X))
[5] ✓ 0.0s
...
layer norm: tensor([[ -1.0000,  1.0000],
                  [-1.0000,  1.0000]], grad_fn=<NativeLayerNormBackward0>)
batch norm: tensor([[ -1.0000, -1.0000],
                   [ 1.0000,  1.0000]], grad_fn=<NativeBatchNormBackward0>)
```

使用残差连接和层规范化来实现 `AddNorm` 类。暂退法也被作为正则化方法使用。

```
class AddNorm(nn.Module):
    def __init__(self, normalized_shape, dropout, **kwargs):
        super(AddNorm, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout)
        self.ln = nn.LayerNorm(normalized_shape)
    def forward(self, X, Y):
        return self.ln(self.dropout(Y) + X)
✓ 0.0s

add_norm = AddNorm([3, 4], 0.5)
add_norm.eval()
add_norm(torch.ones((2, 3, 4)), torch.ones((2, 3, 4))).shape
✓ 0.0s
torch.Size([2, 3, 4])
```

10.7.4 编码器

`EncoderBlock` 类包含两个子层：多头自注意力和基于位置的前馈网络，这两个子层都使用了残差连接和紧随的层规范化。

```
##@save
class EncoderBlock(nn.Module):
    """Transformer编码器块"""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                 norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
                 dropout, use_bias=False, **kwargs):
        super(EncoderBlock, self).__init__(**kwargs)
        self.attention = d2l.MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens, num_heads, dropout,
            use_bias)
        self.addnorm1 = AddNorm(norm_shape, dropout)
        self.ffn = PositionWiseFFN(
            ffn_num_input, ffn_num_hiddens, num_hiddens)
        self.addnorm2 = AddNorm(norm_shape, dropout)

    def forward(self, X, valid_lens):
        Y = self.addnorm1(X, self.attention(X, X, X, valid_lens))
        return self.addnorm2(Y, self.ffn(Y))
0.0s
```

Transformer 编码器中的任何层都不会改变其输入的形状。

```

X = torch.ones((2, 100, 24))
valid_lens = torch.tensor([3, 2])
encoder_blk = EncoderBlock(24, 24, 24, 24, [100, 24], 24, 48, 8, 0.5)
encoder_blk.eval()
encoder_blk(X, valid_lens).shape

torch.Size([2, 100, 24])

```

下面代码堆叠了 `num_layers` 个 `EncoderBlock` 类的实例。由于这里使用的是值范围在 -1 和 1 之间的固定位置编码，因此通过学习得到的输入的嵌入表示的值需要先乘以嵌入维度的平方根进行重新缩放，然后再与位置编码相加。

```

#@save
class TransformerEncoder(d2l.Encoder):
    """Transformer编码器"""
    def __init__(self, vocab_size, key_size, query_size, value_size,
                 num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
                 num_heads, num_layers, dropout, use_bias=False, **kwargs):
        super(TransformerEncoder, self).__init__(**kwargs)
        self.num_hiddens = num_hiddens
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = d2l.PositionalEncoding(num_hiddens, dropout)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module("block"+str(i),
                                 EncoderBlock(key_size, query_size, value_size, num_hiddens,
                                              norm_shape, ffn_num_input, ffn_num_hiddens,
                                              num_heads, dropout, use_bias))

    def forward(self, X, valid_lens, *args):
        # 因为位置编码值在-1和1之间，
        # 因此嵌入值乘以嵌入维度的平方根进行缩放，
        # 然后再与位置编码相加。
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        self.attention_weights = [None] * len(self.blks)
        for i, blk in enumerate(self.blks):
            X = blk(X, valid_lens)
            self.attention_weights[i] = blk.attention.attention_weights
        return X

```

下面指定超参数来创建一个两层的 `Transformer` 编码器。`Transformer` 编码器输出的形状是（批量大小，时间步数目，`num_hiddens`）。

```

encoder = TransformerEncoder(
    200, 24, 24, 24, 24, [100, 24], 24, 48, 8, 2, 0.5)
encoder.eval()
encoder(torch.ones((2, 100), dtype=torch.long), valid_lens).shape

torch.Size([2, 100, 24])

```

10.7.5 解码器

`Transformer` 解码器也是由多个相同的层组成。在 `DecoderBlock` 类中实现的每个层包含了三个子层：解码器自注意力、“编码器 - 解码器”注意力和基于位置的前馈网络。这些子层也都被残差连接和紧随的层规范化围绕。

```

class DecoderBlock(nn.Module):
    """解码器中第i个块"""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                 norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
                 dropout, i, **kwargs):
        super(DecoderBlock, self).__init__(**kwargs)
        self.i = i
        self.attention1 = d2l.MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens, num_heads, dropout)
        self.addnorm1 = AddNorm(norm_shape, dropout)
        self.attention2 = d2l.MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens, num_heads, dropout)
        self.addnorm2 = AddNorm(norm_shape, dropout)
        self.ffn = PositionWiseFFN(ffn_num_input, ffn_num_hiddens,
                                   num_hiddens)
        self.addnorm3 = AddNorm(norm_shape, dropout)

    def forward(self, X, state):
        enc_outputs, enc_valid_lens = state[0], state[1]
        # 训练阶段，输出序列的所有词元都在同一时间处理，
        # 因此state[2][self.i]初始化为None。
        # 预测阶段，输出序列是通过词元一个接着一个解码的，
        # 因此state[2][self.i]包含着直到当前时间步第i个块解码的输出表示
        if state[2][self.i] is None:
            key_values = X
        else:
            key_values = torch.cat((state[2][self.i], X), axis=1)
        state[2][self.i] = key_values
        if self.training:
            batch_size, num_steps, _ = X.shape
            # dec_valid_lens的开头:(batch_size,num_steps),
            # 其中每一行是[1,2,...,num_steps]
            dec_valid_lens = torch.arange(
                1, num_steps + 1, device=X.device).repeat(batch_size, 1)
        else:
            dec_valid_lens = None

        # 自注意力
        X2 = self.attention1(X, key_values, key_values, dec_valid_lens)
        Y = self.addnorm1(X, X2)
        # 编码器-解码器注意力。
        # enc_outputs的开头:(batch_size,num_steps,num_hiddens)
        Y2 = self.attention2(Y, enc_outputs, enc_outputs, enc_valid_lens)
        Z = self.addnorm2(Y, Y2)
        return self.addnorm3(Z, self.ffn(Z)), state

```

为了便于在“编码器—解码器”注意力中进行缩放点积计算和残差连接中进行加法计算，编码器和解码器的特征维度都是 `num_hiddens`。

```

decoder_blk = DecoderBlock(24, 24, 24, 24, [100, 24], 24, 48, 8, 0.5, 0)
decoder_blk.eval()
X = torch.ones((2, 100, 24))
state = [encoder_blk(X, valid_lens), valid_lens, [None]]
decoder_blk(X, state)[0].shape

```

✓ 0.0s

torch.Size([2, 100, 24])

构建一个由 `num_layers` 个 `DecoderBlock` 实例组成的完整的 Transformer 解码器，通过一个全连接层计算所有 `vocab_size` 个可能的输出词元的预测值。解码器的自注意力权重和编码器解码器注意力权重都被存储下来。


```

class TransformerDecoder(d2l.AttentionDecoder):
    def __init__(self, vocab_size, key_size, query_size, value_size,
                 num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
                 num_heads, num_layers, dropout, **kwargs):
        super(TransformerDecoder, self).__init__(**kwargs)
        self.num_hiddens = num_hiddens
        self.num_layers = num_layers
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = d2l.PositionalEncoding(num_hiddens, dropout)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module("block"+str(i),
                                DecoderBlock(key_size, query_size, value_size, num_hiddens,
                                              norm_shape, ffn_num_input, ffn_num_hiddens,
                                              num_heads, dropout, i))
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, enc_valid_lens, *args):
        return [enc_outputs, enc_valid_lens, [None] * self.num_layers]

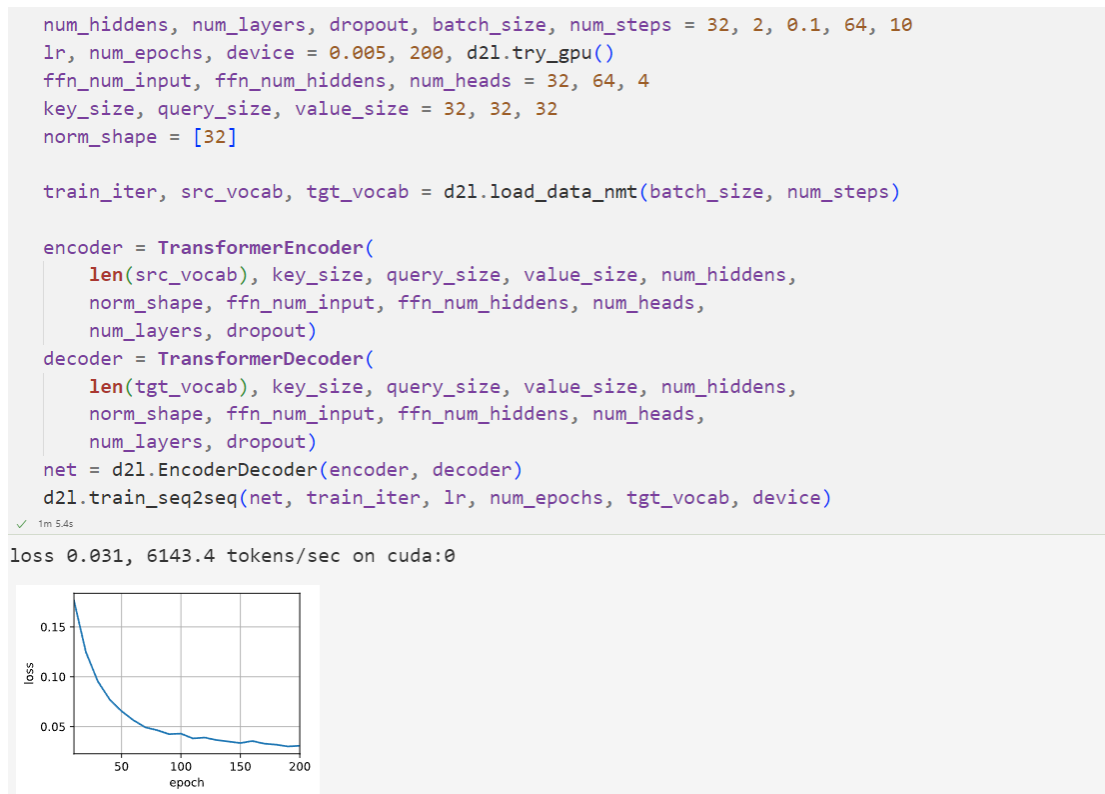
    def forward(self, X, state):
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        self._attention_weights = [[None] * len(self.blks) for _ in range(2)]
        for i, blk in enumerate(self.blks):
            X, state = blk(X, state)
            # 解码器自注意力权重
            self._attention_weights[0][i] = blk.attention1.attention_weights
            # “编码器-解码器”自注意力权重
            self._attention_weights[1][i] = blk.attention2.attention_weights
        return self.dense(X), state

    @property
    def attention_weights(self):
        return self._attention_weights

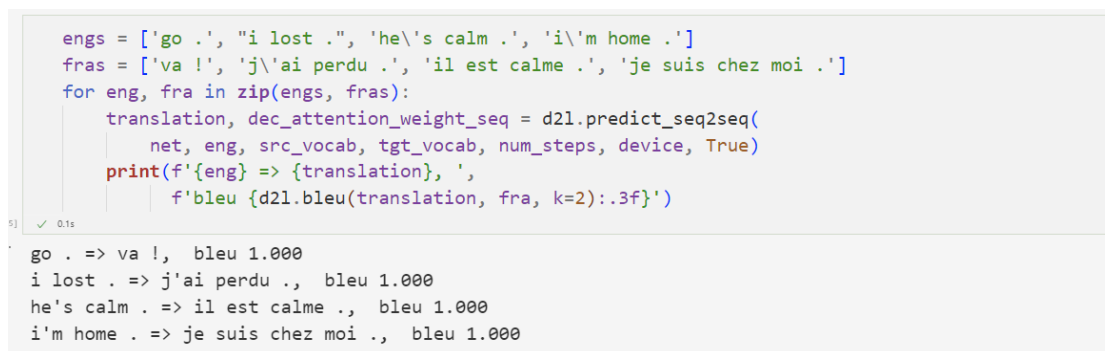
```

10.7.6 训练

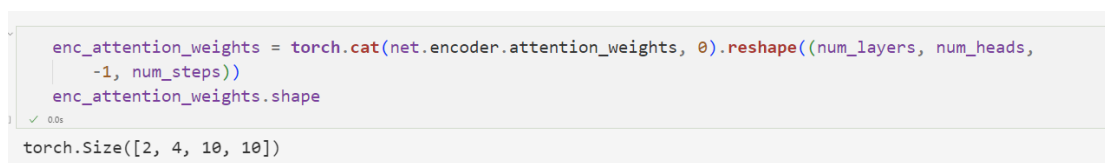
依照 Transformer 架构来实例化编码器-解码器模型，指定 Transformer 的编码器和解码器都是 2 层，都使用 4 头注意力。



训练结束后，使用 Transformer 模型将一些英语句子翻译成法语，并且计算它们的 BLEU 分数。

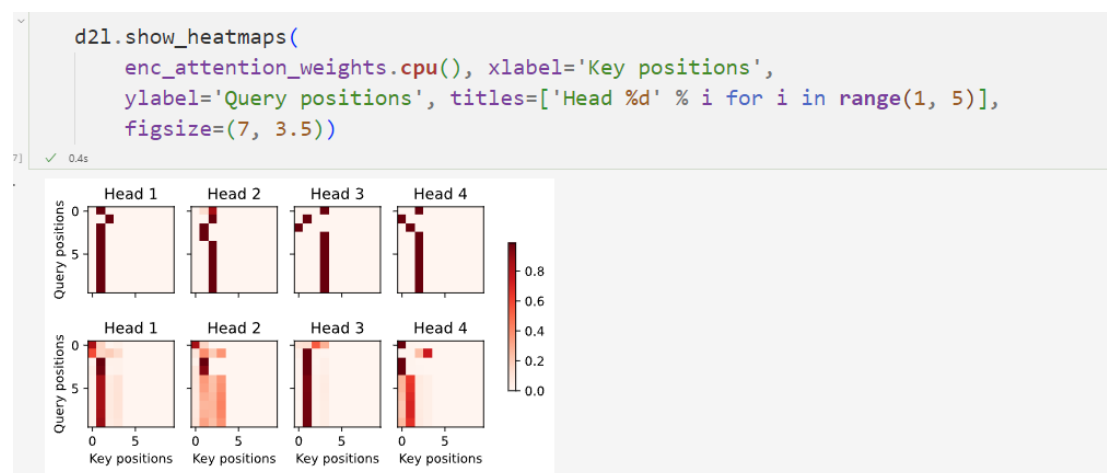


当进行最后一个英语到法语的句子翻译工作时，可视化 Transformer 的注意力权重。编码器自注意力权重的形状为（编码器层数，注意力头数，num_steps 或查询的数目，num_steps 或“键-值”对的数目）。



在编码器的自注意力中，查询和键都来自相同的输入序列。因为填充词元是不携带信息的，因此通过指定输入序列的有效长度可以避免查询与使用填充词元

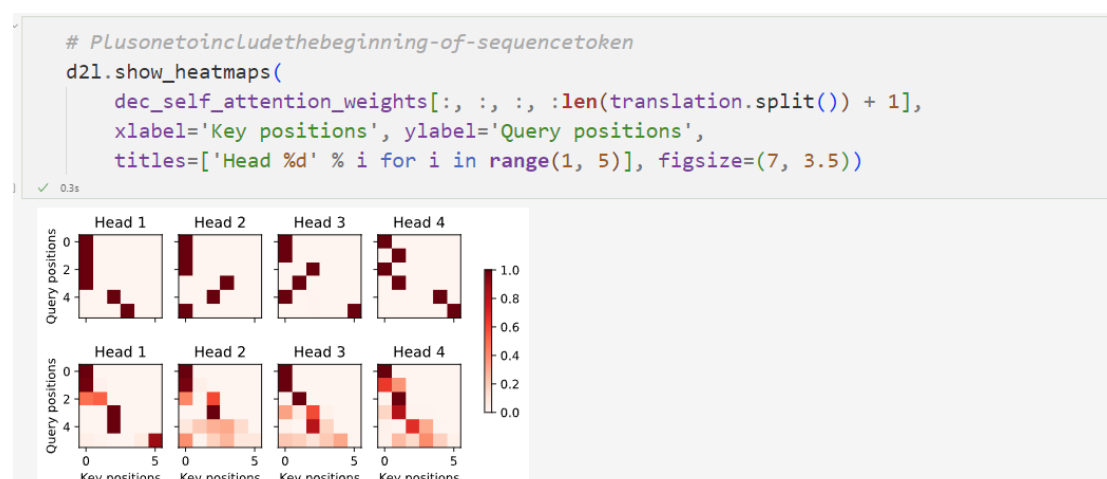
的位置计算注意力。接下来，将逐行呈现两层多头注意力的权重。每个注意力头都根据查询、键和值的不同的表示子空间来表示不同的注意力。



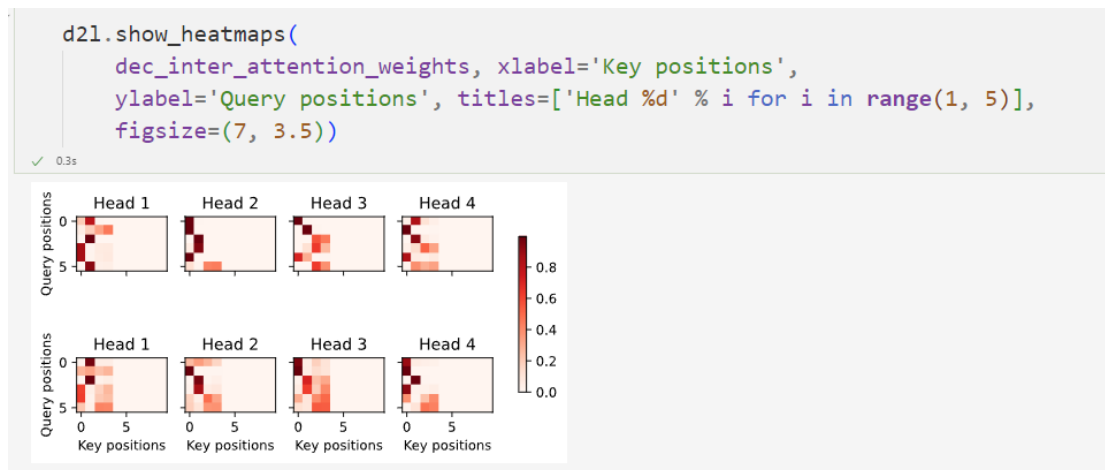
为了可视化解码器的自注意力权重和“编码器—解码器”的注意力权重，可以用零填充被掩蔽住的注意力权重。



由于解码器自注意力的自回归属性，查询不会对当前位置之后的“键—值”对进行注意力计算。



与编码器的自注意力的情况类似，通过指定输入序列的有效长度，输出序列的查询不会与输入序列中填充位置的词元进行注意力计算。



[小结或讨论]

习题

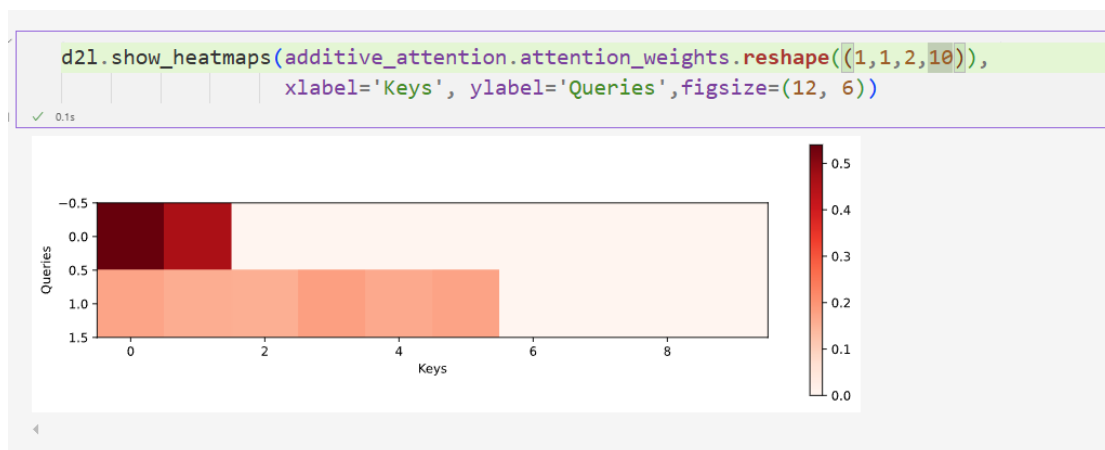
10.3 练习题 1

修改小例子中的键，并且可视化注意力权重。可加性注意力和缩放的“点-积”注意力是否仍然产生相同的结果？为什么？

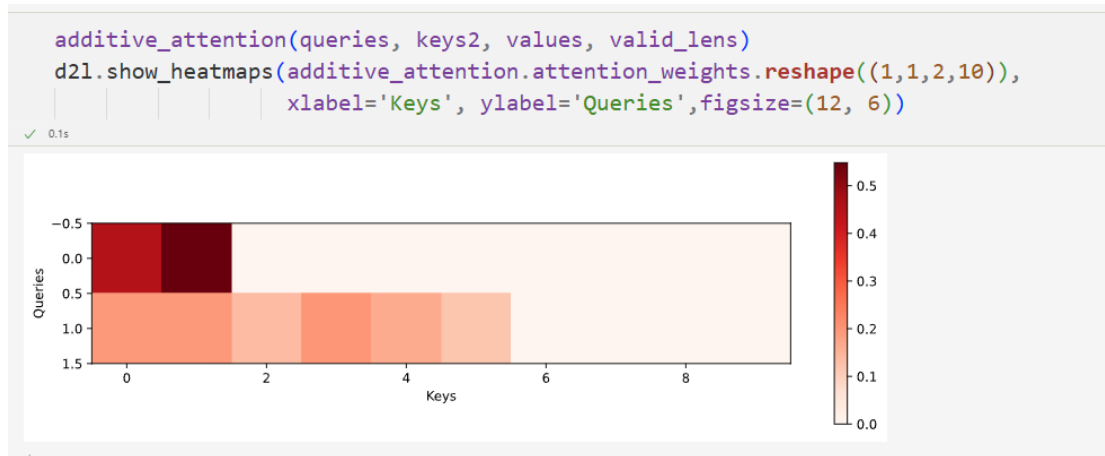
加性注意力：



可视化 Key1

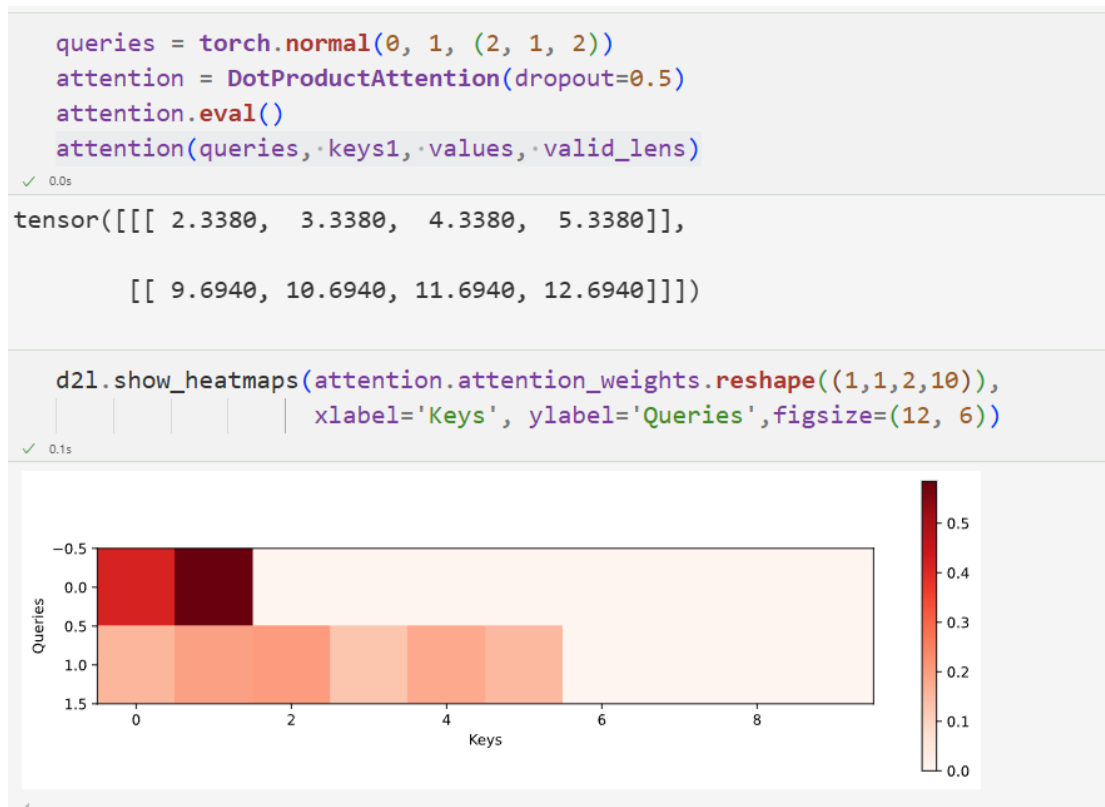


可视化 Key2

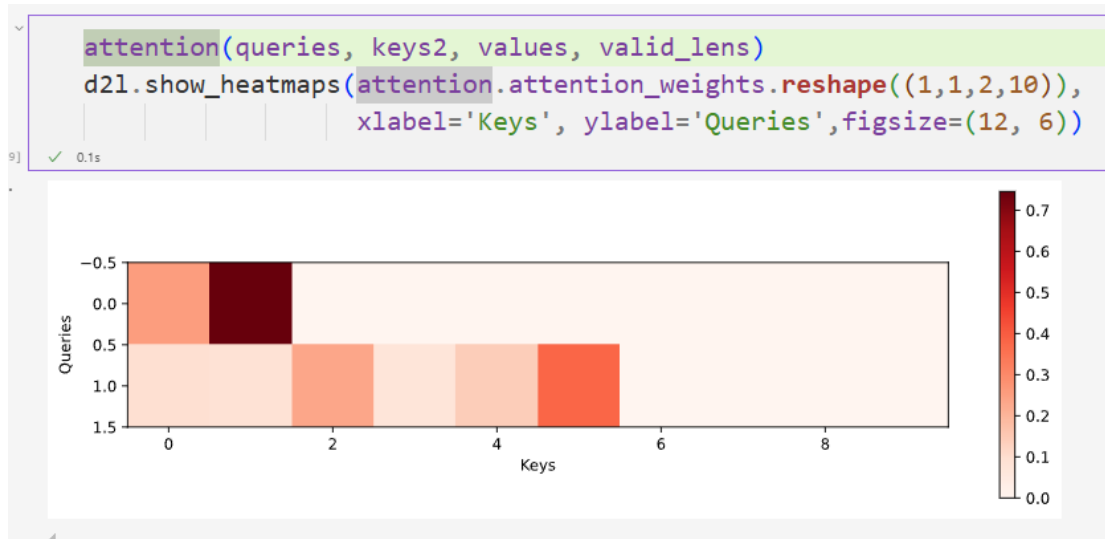


缩放的“点-积”注意力

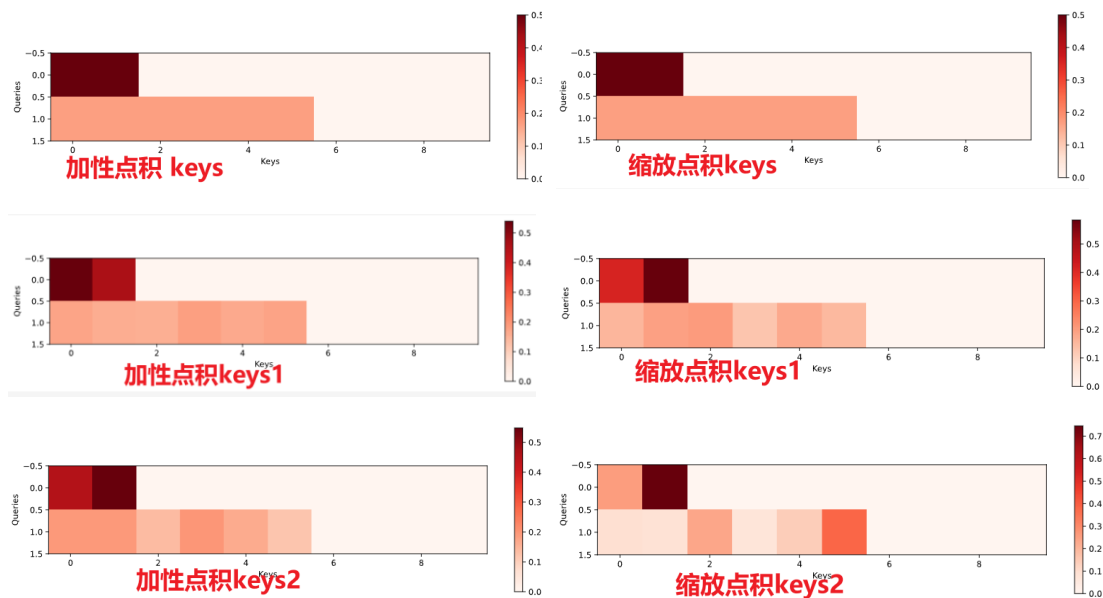
可视化 Key1



可视化 Key2



结果比较:



可加性注意力和缩放的“点-积”注意力产生的结果不同，这可能是因为两种方法计算机制的本质差异

可加性注意力： 通过非线性变换（如 $\tanh$ ）将查询和辅助射到一个中间空间，再计算相关性。

$$\text{公式: } score(q, k) = v^T \tanh(W_q q + W_k k)$$

特点：能够处理查询和一维度不同的情况，引入了非线性变换，对输入的分布变化更简棒。

缩放点积注意力： 直接计算查询和键的点积，再通过缩放因子 $\frac{1}{\sqrt{d}}$ 调整。

$$\text{公式: } score(q, k) = \frac{qk}{\sqrt{d}}$$

特点：计算效率高，但对输入的尺度非常敏感，特别想当维度 d 较大的。

10.5 练习题 1

分别可视化这个实验中的多个头的注意力权重。

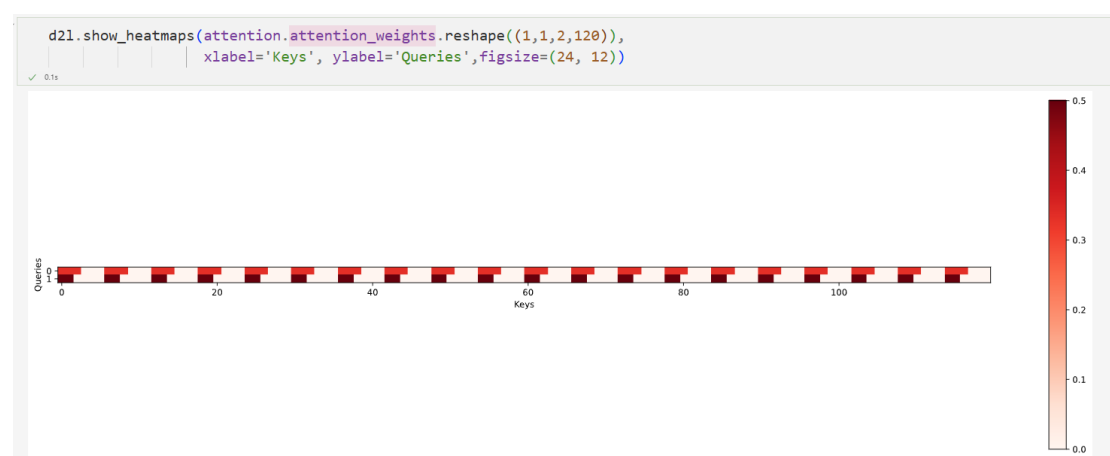
在多头注意力类中加入 `weights` 属性

```
class MultiHeadAttention(nn.Module):
    def __init__(self, key_size, query_size, value_size, num_hiddens, num_heads, dropout, bias=False, **kwargs):
        super(MultiHeadAttention, self).__init__(**kwargs)
        self.num_heads = num_heads
        self.attention = d2l.DotProductAttention(dropout)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=bias)
        self.W_k = nn.Linear(key_size, num_hiddens, bias=bias)
        self.W_v = nn.Linear(value_size, num_hiddens, bias=bias)
        self.W_o = nn.Linear(num_hiddens, num_hiddens, bias=bias)
        self.attention_weights = None # 初始化注意力权重属性

    def forward(self, queries, keys, values, valid_lens):
        queries = transpose_qkv(self.W_q(queries), self.num_heads)
        keys = transpose_qkv(self.W_k(keys), self.num_heads)
        values = transpose_qkv(self.W_v(values), self.num_heads)
        if valid_lens is not None:
            valid_lens = torch.repeat_interleave(valid_lens, repeats=self.num_heads, dim=0)
            # 保存注意力权重, 重塑回[batch_size, num_heads, num_queries, num_kvpairs]

        output = self.attention(queries, keys, values, valid_lens)
        self.attention_weights = self.attention.attention_weights.reshape(2, self.num_heads, queries.shape[1], keys.shape[1])
        output_concat = transpose_output(output, self.num_heads)
        return self.W_o(output_concat)
```

可视化结果：



10.6 习题 2

请设计一种可学习的位置编码方法

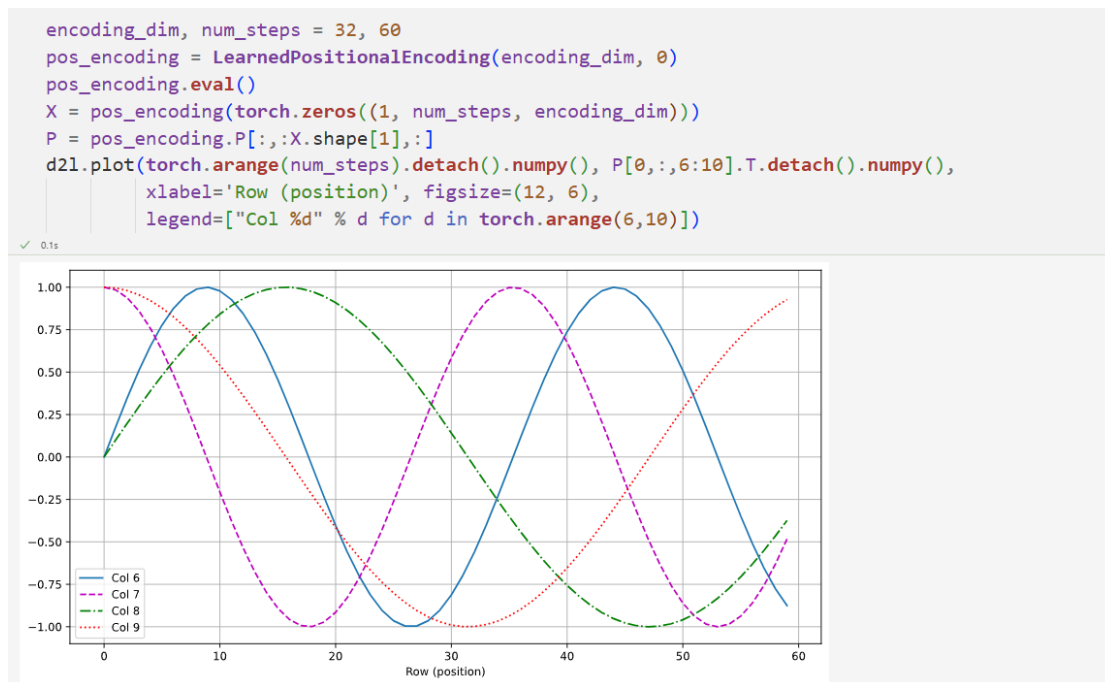
编写可学习的位置编码类

可学习的位置编码实现

```
class LearnedPositionalEncoding(nn.Module):
    """可学习的位置编码实现"""
    (parameter) dropout: Any
    def __init__(self, num_hiddens, dropout, max_len=1000):
        super(LearnedPositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(dropout)
        self.P = torch.zeros((1, max_len, num_hiddens))
        X = torch.arange(max_len, dtype=torch.float32).reshape(-1, 1)/torch.pow(10000, torch.arange(0, num_hiddens, 2, dtype=torch.float32)/num_hiddens)
        self.P[:, :, 0::2] = torch.sin(X)
        self.P[:, :, 1::2] = torch.cos(X)

    def forward(self, X):
        # 切片以适应不同长度的序列
        X = X + self.P[:, :X.shape[1], :].to(X.device)
        return self.dropout(X)
```

实验结果:



与固定的正弦余弦位置编码相比，可学习位置编码具有以下优势：

自适应表示：能够根据具体任务学习最适合的位置表示，而不是依赖预设的数学模式

捕捉复杂模式：可以学习到更复杂的位置依赖关系，特别是在长序列场景中

任务特定优化：编码模式会根据任务目标进行优化，可能在某些场景下表现更好

避免固定模式限制：不受固定正弦余弦模式的限制，可能发现更有效的位置表示方式

10.7 习题 3

对于语言模型，应该使用 **Transformer** 的编码器还是解码器，或者两者都用？如何设计？

在语言模型的设计中，**Transformer** 的编码器和解码器各有适用场景，具体选择

需根据任务需求

仅使用编码器

适用场景：文本理解、特征提取、双向语言模型（如 BERT、RoBERTa）。

设计要点：输入为 token 序列，通过词嵌入和位置编码生成特征。

多层编码器块堆叠，增强上下文捕捉能力。输出可用于下游任务（如分类、问答），或通过掩码语言模型（MLM）预训练。

设计代码：

```
class EncoderOnlyLM(nn.Module):
    def __init__(self, vocab_size, num_hiddens, num_layers, num_heads):
        super().__init__()
        self.encoder = TransformerEncoder(
            vocab_size, key_size=num_hiddens, query_size=num_hiddens,
            value_size=num_hiddens, num_hiddens=num_hiddens,
            norm_shape=[num_hiddens], ffn_num_input=num_hiddens,
            ffn_num_hiddens=4*num_hiddens, num_heads=num_heads,
            num_layers=num_layers, dropout=0.1)
        self.classifier = nn.Linear(num_hiddens, vocab_size)

    def forward(self, X, valid_lens):
        encoded = self.encoder(X, valid_lens)
        # 取CLS token或平均池化用于预测
        return self.classifier(encoded[:, 0, :]) # 假设第一个token为CLS
```

仅使用解码器

适用场景：文本生成、自回归语言模型（如 GPT、GPT-4）。

设计要点：移除编码器 - 解码器注意力，仅保留自注意力（因果掩码确保单向依赖）。输入为已生成的词元序列，通过自回归方式逐个预测下一个词（如代码中 DecoderBlock 的 state 状态传递）。预训练采用自回归语言模型（ARLM）目标，即给定前文预测下一词。

```

class DecoderOnlyLM(d2l.AttentionDecoder):
    def __init__(self, vocab_size, num_hiddens, num_layers, num_heads):
        super().__init__()
        self.num_hiddens = num_hiddens
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = d2l.PositionalEncoding(num_hiddens, dropout=0.1)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            # 注意：移除编码器-解码器注意力，仅保留自注意力
            self.blks.add_module(f"block{i}",
                DecoderBlock(
                    key_size=num_hiddens, query_size=num_hiddens,
                    value_size=num_hiddens, num_hiddens=num_hiddens,
                    norm_shape=[num_hiddens], ffn_num_input=num_hiddens,
                    ffn_num_hiddens=4*num_hiddens, num_heads=num_heads,
                    dropout=0.1, i=i
                ))
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def forward(self, X, state):
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        for blk in self.blks:
            # 仅使用自注意力 (state中无需编码器输出)
            X, state = blk(X, [None, None, state[2]])
        return self.dense(X), state

```

编码器 - 解码器架构

适用场景：序列到序列任务（如机器翻译、文本摘要），需同时处理输入序列和生成输出序列。

设计要点：编码器处理源序列，解码器结合编码器输出生成目标序列。解码器的编码器 - 解码器注意力机制（如 attention2）用于对齐源序列和目标序列。训练时采用教师强制（Teacher Forcing），预测时自回归生成。

```

class Seq2SeqTransformer(d2l.EncoderDecoder):
    def __init__(self, src_vocab_size, tgt_vocab_size, num_hiddens, num_layers, num_heads):
        encoder = TransformerEncoder(
            src_vocab_size, key_size=num_hiddens, query_size=num_hiddens,
            value_size=num_hiddens, num_hiddens=num_hiddens,
            norm_shape=[num_hiddens], ffn_num_input=num_hiddens,
            ffn_num_hiddens=4*num_hiddens, num_heads=num_heads,
            num_layers=num_layers, dropout=0.1)
        decoder = TransformerDecoder(
            tgt_vocab_size, key_size=num_hiddens, query_size=num_hiddens,
            value_size=num_hiddens, num_hiddens=num_hiddens,
            norm_shape=[num_hiddens], ffn_num_input=num_hiddens,
            ffn_num_hiddens=4*num_hiddens, num_heads=num_heads,
            num_layers=num_layers, dropout=0.1)
        super().__init__(encoder, decoder)

```

10.7 习题 6

如果不使用卷积神经网络，如何设计基于 Transformer 模型的图像分类任务？

提示：可以参考 Vision Transformer

ViT 通过将图像转换为序列输入，并利用 Transformer 的自注意力机制捕捉全局语义关系，彻底摒弃了卷积操作。

设计流程：

输入图像： $224 \times 224 \times 3 \rightarrow$ 分块为 14×14 个 $16 \times 16 \times 3$ 的 Patch ($P=16$)。

Patch Embedding：每个 Patch 通过线性层映射为 $D=768$ 维向量，得到 196×768 的序列。

添加分类 Token：在序列开头插入 1×768 的 Class Token，形成 197×768 的输入。

位置编码：生成 197×768 的位置编码，与输入相加。

Transformer 编码：通过 L 层编码器（如 $L=12$ ），每层包含 MHA 和 FFN，输出 197×768 的特征序列。

提取分类特征：取 Class Token 的特征向量 1×768 。

分类预测：通过 MLP 映射到类别数，经 Softmax 输出概率分布。

```
class VisionTransformer(nn.Module):
    """Vision Transformer 图像分类模型"""
    def __init__(self, img_size=224, patch_size=16, in_channels=3,
                 num_classes=1000, embed_dim=768, num_heads=12,
                 num_layers=12, hidden_dim=3072, dropout=0.1):
        super(VisionTransformer, self).__init__()
        # 图像分块嵌入
        self.patch_embed = PatchEmbedding(img_size, patch_size, in_channels, embed_dim)
        num_patches = self.patch_embed.num_patches
        # 分类Token
        self.class_token = nn.Parameter(torch.zeros(1, 1, embed_dim))
        # 位置编码
        self.pos_embed = PositionalEncoding(num_patches + 1, embed_dim, dropout)
        # Transformer 编码器
        self.encoders = nn.ModuleList([
            EncoderBlock(embed_dim, num_heads, hidden_dim, dropout)
            for _ in range(num_layers)
        ])
        # 层归一化
        self.ln = nn.LayerNorm(embed_dim)
        # 分类头
        self.classifier = nn.Linear(embed_dim, num_classes)

    def forward(self, x):
        batch_size = x.shape[0]
        # 图像分块嵌入
        x = self.patch_embed(x) # [batch_size, num_patches, embed_dim]
        # 添加分类Token
        class_tokens = self.class_token.expand(batch_size, -1, -1)
        x = torch.cat((class_tokens, x), dim=1) # [batch_size, num_patches+1, embed_dim]
        # 添加位置编码
        x = self.pos_embed(x) # [batch_size, num_patches+1, embed_dim]
        # Transformer 编码
        for encoder in self.encoders:
            x = encoder(x)
        # 层归一化
        x = self.ln(x) # [batch_size, num_patches+1, embed_dim]
        # 提取分类Token特征
        cls_features = x[:, 0, :] # [batch_size, embed_dim]
        # 分类预测
        out = self.classifier(cls_features) # [batch_size, num_classes]
        return out
```

讨论与收获

1. 注意力机制核心认知

- 1.加性与点积注意力的本质差异：加性注意力通过非线性变换处理不同维度输入，对数据分布更鲁棒；点积注意力依赖直接点积，需缩放因子稳定梯度，修改键矩阵后两者权重分布明显不同。
- 2.多头注意力的并行优势：不同注意力头可捕捉不同语义关系（如局部依赖与全局关联），可视化显示各头关注区域不同，提升模型表征能力。

2. Transformer 关键组件应用

- 1.编码器与解码器的任务适配：编码器适用于双向语义理解（如 BERT），解码器适合自回归生成（如 GPT），编码器-解码器架构用于序列转换（如机器翻译）。
- 2.位置编码的必要性：固定正弦余弦编码适用于长序列，可学习编码能自适应任务需求，两者均通过与词嵌入相加注入位置信息，提升序列顺序建模能力。

3. 编程体会

1.层归一化 vs 批量归一化：层归一化更适合变长序列（如 NLP），不依赖批量大小，稳定性更高。

2.因果掩码的作用：解码器自注意力中通过因果掩码防止解码时访问未来信息，确保生成逻辑合理性。

深度学习模型设计需紧扣任务需求（如生成任务选解码器，理解任务选编码器），注意力机制的灵活性使其在跨领域（NLP、CV）中均表现优异，实验中需注重理论推导与可视化验证结合。